

Đây là file quan trọng nhất. Trong file này, bạn sẽ trình bày chi tiết từng bước (step-by-step) cách áp dụng kỹ thuật Domain Testing và Boundary Value Analysis cho 4 chức năng đã chọn (FR-02, FR-08, FR-13, FR-20). Bạn cũng sẽ ghi nhận phần AI Gap Analysis (những gì AI bỏ sót) tại đây.

Main Report

MSSV: 23127125

Họ và tên: Nguyễn Hiếu Thuận

Bài tập: HW02 - Domain Testing

FR-02: Đăng nhập & Khóa tài khoản

Domain Testing

Để thực hiện Domain Testing cho FR-02, trước tiên tôi tiến hành xác định các biến đầu vào (Email, Mật khẩu) và biến trạng thái (Số lần sai). Sau đó, tôi phân chia miền giá trị hợp lệ/không hợp lệ cho từng biến dựa trên ràng buộc của HTML5 và logic của backend.

Phân tích Miền giá trị (Domain Analysis): Dựa trên đặc tả hệ thống, chức năng Đăng nhập bao gồm 2 biến đầu vào (Email, Password), 1 biến trạng thái (Số lần đăng nhập sai) và các kết quả đầu ra tương ứng.

- Biến đầu vào:** Email (Input Variable): Biến này chịu sự ràng buộc của validate HTML5 type="email" ở Frontend và kiểm tra tồn tại ở Backend.

Miền hợp lệ (Valid Domains):

- V1: Email đúng định dạng chuẩn HTML5 (VD: user@example.com) VÀ đã được đăng ký trong hệ thống.

Miền không hợp lệ (Invalid Domains):

- I1 (Empty): Bỏ trống trường Email.
- I2 (Invalid Format): Email sai định dạng HTML5 (VD: thiếu @, thiếu tên miền như userexample.com, user@.com).
- I3 (Unregistered): Email đúng định dạng HTML5 nhưng chưa được đăng ký trong hệ thống.

- Biến đầu vào:** Mật khẩu (Input Variable): Biến này được đánh giá thông qua việc đối chiếu với cơ sở dữ liệu dựa trên Email hợp lệ.

Miền hợp lệ (Valid Domains):

- V2: Chuỗi ký tự khớp chính xác 100% (có phân biệt hoa/thường) với mật khẩu đã lưu của Email.

Miền không hợp lệ (Invalid Domains):

- I4 (Empty): Bỏ trống trường Mật khẩu.
- I5 (Mismatch): Chuỗi ký tự không khớp với mật khẩu của Email tương ứng.

- Biến trạng thái:** Số lần đăng nhập sai (State Variable): Đây là biến ẩn quyết định luồng khóa tài khoản. Phụ thuộc vào Bộ đếm (Counter) và Thời gian chờ (Timeout).

Miền hợp lệ - Trạng thái Hoạt động (Active State):

- S1: Attempts = 0 (Đăng nhập lần đầu hoặc đã reset bộ đếm sau khi đăng nhập thành công).
- S2: Attempts = 1 hoặc 2 (Đã sai 1-2 lần liên tiếp, tài khoản vẫn được phép thử tiếp).
- S3: Attempts >= 3 VÀ Thời gian từ lần sai cuối >= 30 giây (Đã hết thời gian phạt, tài khoản được mở khóa).

Miền không hợp lệ - Trạng thái Tạm khóa (Locked State):

- S4: Attempts >= 3 VÀ Thời gian từ lần sai cuối < 30 giây (Tài khoản đang bị khóa, từ chối mọi yêu cầu đăng nhập).

4. Miền kết quả kỳ vọng (Expected Output Domains): Dựa trên tổ hợp của các miền giá trị trên, hệ thống sẽ trả về các kết quả sau:

- O1 (Thành công): Xảy ra khi (V1 + V2) + (S1 hoặc S2 hoặc S3). Trạng thái: Trả về JWT Token, reset bộ đếm số lần sai về 0.
- O2 (Lỗi Validation UI): Xảy ra khi I1, I4 hoặc I2. Trạng thái: Frontend chặn submit, hiển thị thông báo yêu cầu nhập đúng định dạng.
- O3 (Lỗi Thông tin - Bảo mật): Xảy ra khi (I3 hoặc I5) + (S1 hoặc S2 hoặc S3). Trạng thái: Hệ thống tăng bộ đếm lên 1. Trả về thông báo lỗi chung chung (VD: "Email hoặc mật khẩu không chính xác") để không để lộ chi tiết nguyên nhân.
- O4 (Lỗi Tạm khóa): Xảy ra khi vi phạm S4. Trạng thái: Trả về thông báo tài khoản đang bị tạm khóa, không thực hiện xác thực Email/Password.

Thiết kế Test Case (Domain Testing) Dựa trên các miền giá trị đã phân tích, dưới đây là tập hợp các Test Case đại diện nhằm đảm bảo bao phủ toàn bộ các kịch bản kết quả (O1, O2, O3, O4) mà không gây ra dư thừa tổ hợp:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-02_01	Kiểm tra đăng nhập thành công	Email: test@eshop.com Pass: Test1234! State: 0 lần sai	Nhập Email, Mật khẩu và gửi yêu cầu đăng nhập.	Đăng nhập thành công, trả về JWT Token.	Đăng nhập thành công.	Pass
TC_FR-02_02	Kiểm tra đăng nhập thành công sau khi hết thời gian phạt	Email: test@eshop.com Pass: Test1234! State: Đang khóa (vừa qua 30s)	Đợi hết thời gian khóa (>30s) và gửi yêu cầu đăng nhập.	Đăng nhập thành công.	Đăng nhập thất bại ở mốc >30s. Thực tế phải chờ qua 50s và reload lại trang (F5) mới đăng nhập được.	Fail
TC_FR-02_03	Kiểm tra validation UI khi bỏ trống Email	Email: (bỏ trống) Pass: Test1234!	Bỏ trống Email, nhập Mật khẩu và click Đăng nhập.	Frontend chặn submit, hiển thị lỗi yêu cầu nhập Email.	Frontend chặn submit, kèm yêu cầu "Please fill out this field." ở ô Username.	Pass
TC_FR-02_04	Kiểm tra validation UI khi Email sai định dạng	Email: testeshop.com Pass: Test1234!	Nhập Email sai định dạng, nhập Mật khẩu và click Đăng nhập.	Frontend chặn submit, hiển thị lỗi định dạng HTML5.	Request vẫn được gửi đi. Frontend hiện thông báo từ Backend: "Đăng nhập thất bại, vui lòng kiểm tra lại".	Fail
TC_FR-02_05	Kiểm tra validation UI khi bỏ trống Password	Email: test@eshop.com Pass: (bỏ trống)	Nhập Email, bỏ trống Mật khẩu và	Frontend chặn submit, hiển thị lỗi yêu cầu	Frontend chặn submit, kèm yêu cầu "Please fill out	Pass

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
			click Đăng nhập.	nhập Mật khẩu.	this field." ở ô Mật khẩu.	
TC_FR-02_06	Kiểm tra lỗi bảo mật khi Email chưa đăng ký	Email: new@example.com Pass: AnyPass123	Nhập thông tin tài khoản chưa tồn tại và gửi yêu cầu.	Trả về lỗi chung ("Email hoặc mật khẩu không đúng").	Frontend hiện: "Đăng nhập thất bại, vui lòng kiểm tra lại".	Pass
TC_FR-02_07	Kiểm tra lỗi bảo mật khi sai Password và đẩy lên giới hạn khóa	Email: test@eshop.com Pass: WrongPass! State: Đã sai 2 lần	Gửi yêu cầu đăng nhập sai lần thứ 3.	Trả về lỗi chung, tăng bộ đếm lên 3 (tạm khóa).	Hệ thống khóa tài khoản ngay ở lần đăng nhập thất bại thứ 2. UI không có thông báo khóa.	Fail
TC_FR-02_08	Kiểm tra từ chối đăng nhập khi tài khoản đang bị khóa, dù nhập đúng	Email: test@eshop.com Pass: Test1234! State: Đang khóa	Gửi yêu cầu đăng nhập với thông tin đúng.	Trả về thông báo tài khoản đang bị tạm khóa.	Chỉ hiện thông báo chung: "Đăng nhập thất bại, vui lòng kiểm tra lại". Không thông báo bị khóa.	Fail
TC_FR-02_09	Kiểm tra từ chối đăng nhập khi tài khoản đang bị khóa, nhập sai	Email: fake@example.com Pass: FakePass State: Đang khóa	Gửi yêu cầu đăng nhập với thông tin sai.	Trả về thông báo tài khoản đang bị tạm khóa.	Chỉ hiện thông báo chung: "Đăng nhập thất bại, vui lòng kiểm tra lại". Không thông báo bị khóa.	Fail

Boundary Value Analysis (BVA)

Phân tích các giá trị biên (Step-by-step Explanation): Tôi tập trung vào 2 ranh giới quyết định việc thay đổi trạng thái của hệ thống: số lần đăng nhập sai và thời gian tạm khóa tài khoản.

1. Ranh giới số lần đăng nhập sai (Ngưỡng n = 3):
- Just below the boundary (n = 2): Kiểm tra khi người dùng nhập sai lần thứ 2. Hệ thống phải ghi nhận lỗi nhưng tài khoản vẫn ở trạng thái hoạt động.
 - On the boundary (n = 3): Kiểm tra khi người dùng nhập sai ở đúng lần thứ 3. Đây là điểm trigger kích hoạt trạng thái tạm khóa.
 - Just above the boundary (n = 4): Kiểm tra khi người dùng tiếp tục gửi request đăng nhập lần thứ 4 (khi tài khoản vừa bị khóa). Hệ thống phải lập tức từ chối mà không cần kiểm tra DB.
2. Ranh giới thời gian tạm khóa (Ngưỡng t = 30s):
- Just below the boundary (t = 29s): Cố gắng thực hiện đăng nhập khi thời gian phạt chưa kết thúc (còn thiếu 1 giây). Hệ thống vẫn phải hiển thị lỗi bị khóa.
 - On the boundary (t = 30s): Cố gắng thực hiện đăng nhập ngay tại thời điểm mốc phạt vừa chạm tới. Hệ thống phải cho phép xác thực (mở khóa tài khoản).
 - Just above the boundary (t = 31s): Cố gắng đăng nhập khi thời gian phạt đã qua mức an toàn. Hệ thống phải tiếp nhận request bình thường.

Thiết kế Test Case (BVA Testing): Dưới đây là các Test Case chi tiết để kiểm thử các giá trị biên đã phân tích:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-02_BVA_01	Kiểm tra biên dưới số lần sai (n = 2)	Tài khoản hợp lệ Email: test@eshop.com Pass: WrongPass! State: Đã sai 1 lần	Gửi yêu cầu đăng nhập sai lần thứ 2.	Hệ thống báo lỗi thông tin không đúng, tăng bộ đếm lên 2, tài khoản không bị khóa.	Hệ thống khóa tài khoản ngay ở lần đăng nhập thất bại thứ 2. Frontend chỉ hiện: "Đăng nhập thất bại, vui lòng kiểm tra lại".	Fail
TC_FR-02_BVA_02	Kiểm tra tại biên số lần sai (n = 3)	Tài khoản hợp lệ Email: test@eshop.com Pass: WrongPass! State: Đã sai 2 lần	Gửi yêu cầu đăng nhập sai lần thứ 3.	Hệ thống báo lỗi tài khoản bị tạm khóa 30s, tăng bộ đếm lên 3.	Chỉ hiện thông báo chung: "Đăng nhập thất bại, vui lòng kiểm tra lại". (Tài khoản thực tế đã bị khóa từ lần 2).	Fail
TC_FR-02_BVA_03	Kiểm tra vượt biên số lần sai (n = 4)	Tài khoản vừa bị khóa Email: test@eshop.com Pass: AnyPass123 State: Đã sai 3 lần	Gửi yêu cầu đăng nhập lần thứ 4 (khi vừa bị khóa).	Hệ thống từ chối đăng nhập ngay lập tức, báo lỗi tài khoản đang bị tạm khóa.	Chỉ hiện thông báo chung: "Đăng nhập thất bại...", không thông báo báo bị khóa.	Fail
TC_FR-02_BVA_04	Kiểm tra biên dưới thời gian khóa (t = 29s)	Tài khoản đang bị khóa Email: test@eshop.com Pass: Test1234! State: Đã sai 3 lần, t = 29s	Gửi yêu cầu đăng nhập tại giây thứ 29.	Đăng nhập thất bại, hệ thống báo lỗi tài khoản vẫn đang bị khóa.	Đăng nhập thất bại. Kể cả F5 lại trang vẫn không đăng nhập được (thực tế phải chờ >50s).	Fail
TC_FR-02_BVA_05	Kiểm tra tại biên thời gian khóa (t = 30s)	Tài khoản đang bị khóa Email: test@eshop.com Pass: Test1234! State: Đã sai 3 lần, t = 30s	Gửi yêu cầu đăng nhập tại đúng giây thứ 30.	Đăng nhập thành công, trả về JWT Token, reset bộ đếm số lần sai về 0.	Đăng nhập thất bại dù đã đúng 30s. Phải chờ qua 50s và tải lại trang (F5) thì mới thành công.	Fail
TC_FR-02_BVA_06	Kiểm tra vượt biên thời gian khóa (t = 31s)	Tài khoản đang bị khóa Email: test@eshop.com Pass: Test1234! State: Đã sai 3 lần, t = 31s	Gửi yêu cầu đăng nhập tại giây thứ 31.	Đăng nhập thành công, trả về JWT Token, reset bộ đếm số lần sai về 0.	Đăng nhập thất bại dù đã qua 31s. Phải chờ qua 50s và tải lại trang (F5) thì mới thành công.	Fail

AI Gap Analysis

1. Sai thông tin tài khoản test:

- AI tạo testcase mặc định với: Email: user@example.com và Password: Abc@123
- Trong khi thực tế môi trường SUT sử dụng tài khoản: test@eshop.com / Test1234!
- **Lý do:** Do câu prompt ban đầu của tôi chỉ tập trung vào logic chức năng mà chưa cung cấp bộ Test Data chuẩn (mock data) của dự án cho AI, dẫn đến AI tự tạo ra data giả.

2. Thiếu format phục vụ Test Execution:

- AI tạo format bảng Test Case bị thiếu 2 cột quan trọng là Kết quả thực tế (Actual Result) và Đánh giá (Verdict).
- **Lý do:** Prompt của tôi yêu cầu AI "thiết kế" (Design) test case chứ chưa yêu cầu chuẩn bị form cho việc "thực thi" (Execution). AI chỉ tuân thủ đúng giới hạn của việc thiết kế.

Bug Reporting

Toàn bộ danh sách lỗi phát hiện được từ các Test Case trên đã được báo cáo chi tiết kèm ảnh chụp màn hình và link GitHub Issues trong file đính kèm Bug_Report.md

FR-08: Thanh toán (Checkout)

Domain Testing

Để thực hiện Domain Testing cho FR-08, tôi không chỉ kiểm tra các thao tác trên giao diện UI (nơi người dùng bị giới hạn) mà tập trung phân tích các biến trạng thái ẩn và dữ liệu Payload API gửi từ Client lên Backend. Mục tiêu là phát hiện các lỗ hổng bảo mật (Data Tampering) và lỗi logic đồng bộ (Race Condition).

Phân tích Miền giá trị (Domain Analysis): Dựa trên đặc tả hệ thống, chức năng Thanh toán phụ thuộc vào 3 nhóm biến trạng thái và đầu vào cốt lõi: Trạng thái xác thực (Auth Token), Trạng thái giỏ hàng (Cart State & Inventory) và dữ liệu Payload thanh toán (total_amount).

1. **Biến trạng thái:** Auth Token (Header Authorization): Trạng thái này quyết định quyền truy cập vào API thanh toán.

Miền hợp lệ (Valid Domains):

- V1: Token tồn tại, đúng định dạng JWT, chữ ký hợp lệ (Valid Signature) và chưa hết hạn.

Miền không hợp lệ (Invalid Domains):

- I1 (Missing): Không gửi kèm Token trong Header Authorization.
- I2 (Expired): Token hợp lệ nhưng đã hết hạn sử dụng.
- I3 (Tampered/Invalid): Token bị chỉnh sửa nội dung, sai chữ ký hoặc là token giả mạo.

2. **Biến trạng thái:** Giỏ hàng (Cart State & Inventory): Trạng thái này xác định tính hợp lệ của dữ liệu đơn hàng trước khi tạo Order.

Miền hợp lệ (Valid Domains):

- V2: Giỏ hàng có ít nhất 1 sản phẩm ($N \geq 1$) và số lượng đặt mua của từng sản phẩm không vượt quá số lượng tồn kho thực tế trong cơ sở dữ liệu.

Miền không hợp lệ (Invalid Domains):

- I4 (Empty Cart): Giỏ hàng trống ($N = 0$) nhưng Client vẫn gửi yêu cầu Checkout.
- I5 (Out of Stock): Tồn kho hiện tại nhỏ hơn số lượng khách hàng đặt mua.
- I6 (Deleted Product): Sản phẩm trong giỏ đã bị Admin xóa hoặc vô hiệu hóa trước thời điểm thanh toán.

3. **Biến đầu vào:** total_amount (Payload API): Đây là trường dữ liệu do Client gửi lên nhưng Backend không được phép tin tưởng hoàn toàn.

Miền hợp lệ (Valid Domains):

- V3: Giá trị total_amount khớp hoàn toàn với tổng tiền Backend tự tính toán từ dữ liệu sản phẩm trong cơ sở dữ liệu hoặc Client không gửi trường này.

Miền không hợp lệ (Invalid Domains):

- I7 (Zero/Negative): total_amount bằng 0 hoặc là số âm.
- I8 (Fractional/Low): total_amount bị sửa thành giá trị rất nhỏ (VD: 1 VNĐ).
- I9 (Overpriced): total_amount bị sửa thành giá trị lớn hơn tổng tiền thực tế.

4. Miền kết quả kỳ vọng (Expected Output Domains): Dựa trên các miền giá trị trên, hệ thống sẽ trả về các kết quả sau:

- O1 (Thanh toán thành công): Xảy ra khi (V1 + V2 + V3). Hệ thống tạo đơn hàng thành công, tính toán chính xác tổng tiền và xóa giỏ hàng sau khi đặt hàng.
- O2 (Lỗi xác thực): Xảy ra khi I1, I2 hoặc I3. Hệ thống từ chối truy cập API và trả về lỗi 401 Unauthorized.
- O3 (Lỗi dữ liệu đơn hàng): Xảy ra khi I4, I5 hoặc I6. Hệ thống từ chối tạo đơn hàng và trả về thông báo phù hợp.
- O4 (Lỗi bảo mật dữ liệu): Xảy ra khi I7, I8 hoặc I9. Backend không được phép sử dụng giá trị total_amount từ Client để tạo đơn hàng sai lệch; phải từ chối hoặc tự tính toán lại từ cơ sở dữ liệu.

Thiết kế Test Case (Domain Testing)

Dựa trên các miền giá trị đã phân tích, dưới đây là tập hợp các Test Case đại diện nhằm đảm bảo bao phủ các kịch bản xác thực, kiểm tra dữ liệu, bảo mật API và xử lý Race Condition:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-08_01	Kiểm tra thanh toán thành công (Happy Path)	Auth: JWT hợp lệ Cart: 2 sản phẩm hợp lệ Payload: total_amount = 58.000.000	Gửi yêu cầu thanh toán hợp lệ với tổng tiền chính xác.	Thanh toán thành công, Backend tính đúng tổng tiền, tạo đơn hàng và xóa giỏ hàng sau khi đặt.	Thanh toán thành công, tuy nhiên giỏ hàng KHÔNG tự xóa. Có thể tiếp tục thanh toán lại. Phải F5 mới xóa giỏ hàng.	Fail
TC_FR-08_02	Kiểm tra bảo mật: Sửa tổng tiền thành 0 VNĐ	Auth: JWT hợp lệ Cart: Sản phẩm trị giá 58.000.000 VNĐ Payload: total_amount = 0	Chỉnh sửa total_amount thành 0 và gửi yêu cầu thanh toán.	Backend bỏ qua giá trị bị sửa, tự tính lại đúng tổng tiền hoặc trả về lỗi 400. Không tạo đơn hàng 0đ.	Backend không validate, vẫn tạo đơn hàng thành công với total_amount = 0.	Fail
TC_FR-08_03	Kiểm tra bảo mật: Sửa tổng tiền thành số âm	Auth: JWT hợp lệ Cart: Sản phẩm trị giá 58.000.000 VNĐ	Chỉnh sửa total_amount thành số âm và gửi yêu cầu thanh toán.	Backend từ chối xử lý với lỗi 400 Bad Request hoặc tự tính lại giá trị hợp lệ.	Backend không validate, vẫn tạo đơn hàng thành công với total_amount = -10.	Fail

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
Payload: total_amount = -10						
TC_FR-08_04	Kiểm tra bảo mật API: Giỏ hàng trống nhưng gọi Checkout	Auth: JWT hợp lệ Cart: 0 sản phẩm	Gửi yêu cầu thanh toán khi giỏ hàng đang rỗng.	Backend trả về lỗi 400, không tạo đơn hàng rỗng.	Sau khi reload page, giỏ hàng rỗng nhưng form báo tổng tiền là 1 (hoặc 0). Bấm thanh toán vẫn tạo đơn hàng trống thành công.	Fail
TC_FR-08_05	Kiểm tra Race Condition: Hết hàng lúc thanh toán	Auth: JWT hợp lệ Cart: Có sản phẩm A Tồn kho A = 0	Gửi yêu cầu thanh toán.	Thanh toán thất bại, thông báo sản phẩm không đủ số lượng hoặc đã hết hàng.	Hệ thống không quản lý tồn kho, bỏ qua test case này.	N/A
TC_FR-08_06	Kiểm tra Race Condition: Sản phẩm bị xóa trước thanh toán	Auth: JWT hợp lệ Cart: Có sản phẩm B Sản phẩm B đã bị xóa khỏi hệ thống	Gửi yêu cầu thanh toán.	Thanh toán thất bại, thông báo sản phẩm không còn tồn tại.	Trong giỏ hàng vẫn hiển thị sản phẩm B. Tiến hành thanh toán thì vẫn hiện thanh toán thành công.	Fail
TC_FR-08_07	Kiểm tra quyền truy cập: Không có Token	Cart: Có sản phẩm hợp lệ Auth: Không đính kèm Token	Gửi yêu cầu API thanh toán không có Authorization Header.	Backend trả về lỗi 401 Unauthorized.	Backend không trả về 401. Trình duyệt vắng lỗi CORS policy và chặn request do lỗi từ Middleware.	Fail
TC_FR-08_08	Kiểm tra quyền truy cập: Token không hợp lệ/hết hạn	Cart: Có sản phẩm hợp lệ Auth: JWT sai chữ ký / hết hạn	Gửi yêu cầu API thanh toán với Token không hợp lệ.	Backend trả về lỗi 401 Unauthorized. Frontend chuyển hướng về trang đăng nhập.	Backend không trả về 401. Trình duyệt vắng lỗi CORS policy và chặn request do lỗi từ Middleware.	Fail
TC_FR-08_09	Kiểm tra chặn thanh toán UI khi chưa đăng nhập	Auth: Chưa đăng nhập Cart: Có sản phẩm trong giỏ	Click nút thanh toán trên giao diện.	Frontend chặn thao tác, tự động chuyển hướng (redirect) về trang Login.	Hệ thống hiển thị thông báo lỗi yêu cầu đăng nhập và chuyển hướng về trang đăng nhập đúng như kỳ vọng.	Pass

Boundary Value Analysis (BVA)

Phân tích các giá trị biên (Step-by-step Explanation): Dựa trên đặc tả hệ thống và việc giao diện UI không chặn độ dài của trường "Số lượng" (N), tôi tập trung phân tích 2 ranh giới cực độ có khả năng gây lỗi (Crash hoặc tạo đơn hàng ảo) tại API Checkout. Do hệ thống không quản lý tồn kho, mọi số lượng $N > 0$ đều được coi là hợp lệ về mặt logic nghiệp vụ, đẩy toàn bộ rủi ro về phía xử lý kiểu dữ liệu của Backend.

1. Ranh giới giới hạn tối thiểu của Giỏ hàng (Ngưỡng $N = 1$):
- Just below the boundary ($N = 0$): Kiểm tra khi người dùng cố tình nhập số lượng là 0. Hệ thống phải từ chối thanh toán (tương đương giỏ hàng rỗng hoặc số lượng không hợp lệ).
 - On the boundary ($N = 1$): Kiểm tra khi người dùng mua đúng 1 sản phẩm. Đây là mốc hợp lệ tối thiểu, hệ thống phải xử lý thành công.
 - Just above the boundary ($N = 2$): Kiểm tra khi người dùng mua 2 sản phẩm, hệ thống phải xử lý tính toán tổng tiền chính xác và thanh toán thành công.
2. Ranh giới giới hạn lưu trữ tổng tiền (Integer Overflow - Ngưỡng Max Int 32-bit): Với kiểu dữ liệu số nguyên 32-bit có dấu, giá trị lớn nhất lưu trữ được là 2.147.483.647. Sản phẩm "iPhone 15 Pro Max" có giá 30.000.000 VNĐ. Ta phân tích quanh ranh giới gây tràn số:
- Just below the boundary ($N = 71$): Tổng tiền là $71 * 30.000.000 = 2.130.000.000$ VNĐ (Năm sát dưới mức Max Int). Hệ thống vẫn phải xử lý thành công mà không bị sai lệch số liệu.
 - Just above the boundary / On Overflow ($N = 72$): Tổng tiền là $72 * 30.000.000 = 2.160.000.000$ VNĐ (Vượt mức Max Int). Việc truyền giá trị này có thể khiến Backend bị crash, lưu sai dữ liệu (biến thành số âm), hoặc báo lỗi Server (500). Kỳ vọng đúng là hệ thống sử dụng kiểu dữ liệu lớn hơn (như 64-bit/BigInt) hoặc có Validation bắt lỗi "Số tiền vượt quá giới hạn" một cách gọn gàng (Graceful error handling).

Thiết kế Test Case (BVA Testing): Dưới đây là các Test Case chi tiết sử dụng mock data "iPhone 15 Pro Max" (30.000.000 VNĐ) để kiểm thử các giá trị biên:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-08_BVA_01	Kiểm tra biên dưới số lượng ($N = 0$)	Auth: Đã đăng nhập Sản phẩm: iPhone 15 Pro Max Số lượng (N): 0 Payload: <code>total_amount = 0</code>	Gửi yêu cầu thanh toán với số lượng = 0.	Backend từ chối request, trả về lỗi 400 (Số lượng không hợp lệ). Không tạo đơn hàng.	Hệ thống không validate, vẫn cho phép thanh toán thành công đơn hàng có sản phẩm với số lượng 0.	Fail
TC_FR-08_BVA_02	Kiểm tra tại biên số lượng tối thiểu ($N = 1$)	Auth: Đã đăng nhập Sản phẩm: iPhone 15 Pro Max Số lượng (N): 1 Payload: <code>total_amount = 30.000.000</code>	Gửi yêu cầu thanh toán.	Thanh toán thành công, tạo đơn hàng với tổng tiền chính xác là 30.000.000 VNĐ.	Hệ thống hiển thị thanh toán thành công với giá tiền chính xác.	Pass
TC_FR-08_BVA_03	Kiểm tra vượt biên số lượng	Auth: Đã đăng nhập Sản phẩm:	Gửi yêu cầu thanh toán.	Thanh toán thành công, tạo đơn hàng với tổng tiền chính	Hệ thống hiển thị thanh toán thành công với	Pass

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
	tối thiểu (N = 2)	iPhone 15 Pro Max Số lượng (N): 2 Payload: total_amount = 60.000.000		xác là 60.000.000 VNĐ.	giá tiền chính xác.	
TC_FR-08_BVA_04	Kiểm tra sát dưới giới hạn Max Int 32-bit (N = 71)	Auth: Đã đăng nhập Sản phẩm: iPhone 15 Pro Max Số lượng (N): 71 Payload: total_amount = 2.130.000.000	Gửi yêu cầu thanh toán với số lượng và tiền lớn (chưa tràn số).	Thanh toán thành công, tạo đơn hàng với tổng tiền 2.130.000.000 VNĐ. Backend không bị lỗi số học.	Hệ thống hiển thị thanh toán thành công với giá tiền chính xác.	Pass
TC_FR-08_BVA_05	Kiểm tra vượt biên Max Int 32-bit (Gây tràn số)	Auth: Đã đăng nhập Sản phẩm: iPhone 15 Pro Max Số lượng (N): 72 Payload: total_amount = 2.160.000.000	Gửi yêu cầu thanh toán vượt qua ranh giới 32-bit integer.	Hệ thống xử lý an toàn (hỗ trợ BigInt hoặc chặn request). Tuyệt đối không crash server.	Hệ thống hiển thị thanh toán thành công với giá tiền chính xác (Backend không bị tràn số 32-bit).	Pass

AI Gap Analysis

1. Bỏ sót luồng thao tác UI của Guest (Khách vắng lai):

- AI đã thiết kế test case cho việc thiếu Token (TC_FR-08_07) nhưng chỉ ở mức độ gọi thẳng API (Backend). AI bỏ sót hoàn toàn kịch bản thao tác trên giao diện Frontend: Người dùng chưa đăng nhập vẫn có thể thêm sản phẩm vào giỏ và nhìn thấy nút "Tiến hành thanh toán".
- **Lý do:** AI có xu hướng tập trung vào việc bẻ gãy logic API và payload dựa trên đặc tả backend ("Chỉ người dùng đã đăng nhập mới tiến hành thanh toán được") mà thiếu đi ngữ cảnh về luồng trải nghiệm người dùng (UX) thực tế trên trình duyệt.

2. Sinh dữ liệu không thực tế (Hallucination về sản phẩm và giá):

- AI tự động đưa ra các con số về số lượng sản phẩm và giá cả cụ thể (ví dụ: giỏ hàng 500k) trong các testcase.
- **Lý do:** Do trong prompt không cung cấp file chứa mock data hoặc thông tin về các sản phẩm có sẵn, AI đã tự điền dữ liệu giả định để hoàn thiện cấu trúc test case.

3. Tưởng tượng (Hallucination) về tính năng Tồn kho (Stock):

- AI tự thiết kế các test case (TC_FR-08_05, TC_FR-08_06) liên quan đến việc kiểm tra tồn kho (số lượng kho = 0) và Race Condition.
- **Lý do:** Trong các hệ thống E-commerce thực tế luôn có cơ chế tồn kho, nhưng SUT EShop (môi trường demo môn học) lại hoàn toàn không có thuộc tính stock. Đặc tả (business rules) cũng không hề đề cập tới stock. Việc AI mang kinh nghiệm thực tế áp đặt vào hệ thống đã dẫn đến các test case không thể thực thi (N/A).

Bug Reporting

Toàn bộ danh sách lỗi phát hiện được từ các Test Case trên đã được báo cáo chi tiết kèm ảnh chụp màn hình và link GitHub Issues trong file đính kèm [Bug_Report.md](#)

FR-13: Dashboard (Trang chủ)

Domain Testing

Để thực hiện Domain Testing cho FR-13, do chức năng này không có input nhập liệu trực tiếp từ người dùng, tôi tiến hành phân tích đầu vào chính là trạng thái dữ liệu (Data State) của các Đơn hàng (Orders) được lưu trữ trong Database. Hệ thống EShop sẽ truy vấn, gom nhóm và tính toán dựa trên các dữ liệu này.

Phân tích Miền giá trị (Domain Analysis): Dựa trên đặc tả hệ thống, chức năng Dashboard phụ thuộc vào 3 biến trạng thái dữ liệu cốt lõi:

1. **Biến trạng thái:** Số lượng đơn hàng trong DB (Database State): Biến này quyết định việc hiển thị tổng số lượng đơn hàng.

Miền hợp lệ (Valid Domain):

- V1 (Có dữ liệu): $N > 0$ (Hệ thống có chứa ít nhất 1 record đơn hàng).

Miền biên vắng (Empty Domain):

- E1 (Trống): $N = 0$ (Database hoàn toàn trống, chưa có đơn hàng nào được tạo).

2. **Biến trạng thái:** Trạng thái đơn hàng (status): Biến này quyết định điều kiện lọc để tính tổng doanh thu.

Miền hợp lệ tính doanh thu (Revenue-counted Domain):

- V2: status = **delivered** (Đơn hàng đã giao thành công, thỏa điều kiện cộng vào doanh thu).

Miền loại trừ tính doanh thu (Excluded Domain):

- I1: status != **delivered** (VD: pending, processing, cancelled...). Các đơn này chỉ được đếm vào "Tổng số đơn hàng", tuyệt đối không được cộng vào doanh thu.

3. **Biến dữ liệu:** Giá trị đơn hàng (**total_amount**): Biến này là giá trị trực tiếp để cộng dồn vào doanh thu. Dựa trên thực tế hệ thống đã có lỗi hổng từ FR-08, ta có các miền sau:

Miền hợp lệ thông thường (Standard Domain):

- V3: total_amount > 0 (Giá trị đơn hàng là số dương hợp logic).

Miền ngoại lệ (Edge/Bug-derived Domains):

- E2: total_amount = 0 (Đơn hàng có giá 0 VNĐ, sinh ra từ lỗi hổng thiếu validation).
- E3: total_amount < 0 (Đơn hàng có giá trị âm, sinh ra từ lỗi hổng thiếu validation).

Thiết kế Test Case (Domain Testing)

Dựa trên các miền giá trị đã phân tích, dưới đây là tập hợp các Test Case đại diện nhằm đảm bảo bao phủ các kịch bản tính toán, đếm số lượng và xử lý các giá trị dị thường từ DB:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-13_01	Kiểm tra Dashboard khi DB trống hoàn toàn	DB không có bất kỳ record đơn hàng nào.	Truy cập vào trang Dashboard (hoặc gọi API lấy thống kê).	Tổng doanh thu hiển thị 0đ, tổng số đơn hàng hiển thị 0.	Dashboard hiển thị tổng số đơn hàng là 0 và tổng doanh thu là 0đ đúng như kỳ vọng.	Pass
TC_FR-13_02	Kiểm tra doanh thu với 1 đơn thành công	DB có 1 đơn hàng 30.000.000đ (delivered)	Truy cập vào trang Dashboard.	Tổng doanh thu hiển thị 30.000.000đ, tổng số đơn hàng hiển thị 1.	Dashboard hiển thị đúng 1 đơn hàng, nhưng tổng doanh thu bị tính sai (nhân đôi giá trị thực tế) thành 60.000.000đ.	Fail
TC_FR-13_03	Kiểm tra đếm số đơn nhưng không tính doanh thu khi chưa giao	DB có 3 đơn hàng dương tiền nhưng trạng thái: pending, confirmed, shipping.	Truy cập vào trang Dashboard.	Tổng doanh thu hiển thị 0đ, tổng số đơn hàng hiển thị 3.	Dashboard hiển thị tổng số đơn hàng là 3 và tổng doanh thu là 0đ đúng như kỳ vọng.	Pass
TC_FR-13_04	Kiểm tra tính doanh thu với trạng thái hỗn hợp (Happy Path)	DB có 4 đơn hàng: - Đơn 1: 30M (delivered) - Đơn 2: 28M (delivered) - Đơn 3: 6M (pending) - Đơn 4: 45M (canceled)	Truy cập vào trang Dashboard.	Tổng doanh thu tính đúng 58.000.000đ (cộng đơn 1 và 2), tổng số đơn hàng hiển thị 4.	Dashboard đếm đúng 4 đơn hàng, nhưng tổng doanh thu hiển thị 116.000.000đ (Bị nhân đôi).	Fail
TC_FR-13_05	Kiểm tra tính doanh thu chứa đơn hàng 0 VNĐ	DB có 1 đơn hàng 0đ (delivered)	Truy cập vào trang Dashboard.	Tổng doanh thu tính đúng 0đ, tổng số đơn hàng hiển thị 1.	Dashboard hiển thị 1 đơn hàng và doanh thu 0đ. (Pass do $0 \times 2 = 0$).	Pass
TC_FR-13_06	Kiểm tra tính doanh thu chứa đơn hàng giá trị âm	DB có 6 đơn hàng: - Đơn 1: -30.000.000đ (delivered) - Các đơn còn lại: Tổng 58.000.000đ	Truy cập vào trang Dashboard.	Tổng doanh thu tính toán đúng giá trị đại số là 28.000.000đ, tổng số đơn hàng hiển thị 6.	Hệ thống thực hiện tính toán số âm bình thường, nhưng kết quả bị nhân đôi thành 56.000.000đ.	Fail

Boundary Value Analysis (BVA)

Phân tích các giá trị biên (Step-by-step Explanation):

Khác với các form nhập liệu thông thường, giao diện Dashboard không có input trực tiếp. Ranh giới (boundary) ở đây nằm ở giới hạn kiểu dữ liệu của hệ thống Backend/Database và giới hạn hiển thị (layout) của Frontend. Đặc biệt, việc tồn

tại Bug 13 (nhân đôi doanh thu) làm thay đổi hoàn toàn cách chúng ta tiếp cận ranh giới tràn bộ nhớ số nguyên (Integer Overflow).

Dưới đây là phân tích chi tiết cho 3 ranh giới chính:

1. Biên số lượng đơn hàng (N):

- **Min boundary (N = 0):** Kiểm tra trạng thái rỗng khi Database chưa có dữ liệu. Tổng số đơn và tổng doanh thu đều phải là 0.
- **Just above min (N = 1):** Kiểm tra khi hệ thống có đúng 1 đơn hàng đầu tiên.
- **Max boundary (N cực lớn):** Kiểm tra khi số lượng đơn hàng lên tới hàng trăm ngàn hoặc hàng triệu (VD: N = 999.999) xem việc đếm (COUNT) có làm chậm hệ thống hay vỡ UI của thẻ hiển thị số lượng hay không.

2. Biên lưu trữ Tổng doanh thu (Integer Overflow) & Tác động của Bug 13:

- Giới hạn tối đa của kiểu số nguyên 32-bit có dấu (Max Int) là 2.147.483.647.
- Thông thường, ta sẽ test giá trị trong DB mập mé mốc 2,14 tỷ. Tuy nhiên, do Backend đang mắc **Bug 13 (tự động x2 doanh thu)**, ngưỡng tràn số thực tế đối với dữ liệu đầu vào (DB) bị giảm đi một nửa (chỉ còn khoảng 1.073.741.823).
- **Just below boundary:** Tạo đơn hàng trong DB có giá trị 1.050.000.000đ. Khi tính toán, Backend x2 thành 2.100.000.000đ (vẫn nằm trong giới hạn an toàn của 32-bit). Hệ thống không được crash.
- **Just above boundary (Vượt biên):** Tạo đơn hàng trong DB có giá trị 1.080.000.000đ. Backend sẽ x2 thành 2.160.000.000đ. Mức này vượt qua mốc Max Int 32-bit. Cần kiểm tra xem Backend có:
 - Văng lỗi 500 Internal Server Error.
 - Trả về số âm dị thường do tràn bit.
 - Hay đã phòng ngừa bằng cách sử dụng kiểu dữ liệu BigInt/Long (64-bit).

3. Biên hiển thị giao diện (UI Layout):

- **Tràn UI:** Cố tình tạo một tổng doanh thu cực kỳ lớn, dài hơn 15-20 chữ số (VD: 300.000.000.000.000đ - 300 nghìn tỷ) để kiểm tra Frontend xử lý hiển thị như thế nào.
- Text không được phép tràn khỏi Card, phá vỡ bố cục CSS hoặc đè lên các thành phần khác.
- Hệ thống nên tự động format hoặc rút gọn số liệu một cách hợp lý.

Thiết kế Test Case (BVA Testing):

Dưới đây là các Test Case chi tiết để kiểm thử các giá trị biên đã phân tích đối với chức năng Dashboard:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-13_BVA_01	Kiểm tra biên dưới số lượng đơn (N = 0)	DB hoàn toàn trống (0 đơn hàng).	Truy cập Dashboard và kiểm tra hiển thị.	Dashboard hiển thị số lượng: 0. Doanh thu: 0đ.	Dashboard hiển thị tổng số đơn hàng bằng 0 và tổng doanh thu bằng 0đ.	Pass
TC_FR-13_BVA_02	Kiểm tra sát biên dưới số	DB có 1 đơn hàng delivered (Giá: 30.000.000đ).	Truy cập Dashboard và kiểm tra hiển thị.	Dashboard hiển thị số lượng: 1. (Doanh thu x2	Dashboard hiển thị tổng số đơn hàng bằng 1 và tổng	Pass

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
	lượng đơn (N = 1)			thành 60.000.000đ do Bug 13).	doanh thu bằng 60.000.000đ.	
TC_FR-13_BVA_03	Kiểm tra biên cận dưới giới hạn Max Int 32-bit	DB có 1 đơn hàng delivered giá trị: 1.050.000.000đ.	Truy cập Dashboard và kiểm tra tính toán tổng doanh thu.	Hệ thống nhân đôi thành 2.100.000.000đ (chưa vượt Max Int). Không bị lỗi số học hay crash.	Dashboard hiển thị tổng số đơn hàng bằng 1 và tổng doanh thu bằng 2.100.000.000đ. Hệ thống không bị crash.	Pass
TC_FR-13_BVA_04	Kiểm tra vượt biên Max Int 32-bit (Do Bug 13)	DB có 1 đơn hàng delivered giá trị: 1.080.000.000đ.	Truy cập Dashboard và kiểm tra xử lý tràn số.	Mức x2 là 2.160.000.000đ vượt mốc 32-bit. Hệ thống phải dùng kiểu BigInt/Long để hiển thị. Tuyệt đối không crash 500.	Dashboard hiển thị tổng số đơn hàng bằng 1 và tổng doanh thu bằng 2.160.000.000đ. Xử lý an toàn không lỗi.	Pass
TC_FR-13_BVA_05	Kiểm tra giới hạn hiển thị UI (Vỡ Layout Doanh thu)	DB có đơn hàng delivered tổng giá trị: 300.000.000.000.000đ.	Kéo giãn/thu nhỏ giao diện Dashboard.	Giao diện co giãn hợp lý, text không tràn ra ngoài khung Card hay đè lên thẻ khác.	Hiển thị doanh thu bằng 600.000.000.000.000đ. Khi kéo giãn UI số bị tràn ra khỏi box. Không responsive.	Fail
TC_FR-13_BVA_06	Kiểm tra hiển thị UI số lượng đơn hàng (N cực lớn)	DB chứa 999.999.999.999.999 đơn hàng.	Truy cập Dashboard (hoặc can thiệp HTML qua F12 để mock dữ liệu).	Số đếm đơn hàng hiển thị vừa vặn trong Card, có xử lý co chữ hoặc responsive.	Khi kéo giãn UI, chữ số bị tràn ra khỏi box. Lỗi giao diện thẻ Tổng đơn hàng không responsive.	Fail

AI Gap Analysis

1. Tưởng tượng (Hallucination) về trạng thái đơn hàng:

- AI đã tự động đưa trạng thái **processing** vào kịch bản test **TC_FR-13_02**. Tuy nhiên, đối chiếu với đặc tả State Machine của hệ thống (FR-10), vòng đời đơn hàng hoàn toàn không có trạng thái này (chỉ có **pending, confirmed, shipping, delivered, canceled**).
- **Lý do:** AI áp dụng kiến thức chung (general knowledge) về các hệ thống E-commerce thực tế bên ngoài thay vì tuân thủ chặt chẽ ranh giới của hệ thống SUT hiện tại.

2. Sinh dữ liệu không thực tế (Arbitrary Data Generation):

- AI tự động gán các giá trị tiền tệ ngẫu nhiên (1.000.000đ, 200.000đ...) vào các test case để dễ minh họa việc tính tổng.
- **Lý do:** Prompt tập trung vào logic gom nhóm và trạng thái mà không cung cấp mock data cụ thể cho database. Do đó, AI tự fill các con số ngẫu nhiên vào để bảng test case có thể thực thi được.

Bug Reporting

Toàn bộ danh sách lỗi phát hiện được từ các Test Case trên đã được báo cáo chi tiết kèm ảnh chụp màn hình và link GitHub Issues trong file đính kèm [Bug_Report.md](#)

FR-20: Giỏ hàng (Shopping Cart) trên Mobile

Domain Testing

Phân tích Miền giá trị (Domain Analysis):

Đối với FR-20, đầu vào không chỉ là dữ liệu nhập liệu mà còn bao gồm thao tác cảm ứng trên thiết bị mobile. Tôi tách bài toán thành 5 biến chính: thao tác chạm để chọn sản phẩm, thao tác vuốt để xóa sản phẩm, trạng thái số lượng sản phẩm, trạng thái tổng tiền tự động tính lại trên giao diện, và biến trạng thái đăng nhập (Session). Trước hết, giỏ hàng phải có ít nhất một sản phẩm thì các thao tác chọn và xóa mới có ý nghĩa. Tiếp theo, số lượng sản phẩm được cập nhật trực tiếp trên UI nên miền hợp lệ phải bắt đầu từ 1. Do đặc tả không quy định việc giảm số lượng về 0 sẽ xóa sản phẩm, việc cố tình giảm số lượng về 0 (qua UI hoặc API) là một thao tác ngoại lệ (invalid domain) cần kiểm thử để đảm bảo hệ thống chặn lại an toàn ở mức 1.

1. Biến đầu vào: Thao tác chạm (Tap) trên màn hình cảm ứng **Miền hợp lệ (Valid Domains):**

- V1: Chạm đúng vào sản phẩm đang có trong giỏ để chọn sản phẩm.
- V2: Chạm đúng vào vùng điều khiển số lượng để tăng hoặc giảm số lượng. **Miền không hợp lệ (Invalid Domains):**
- I1: Chạm vào vùng trống ngoài danh sách sản phẩm.
- I2: Chạm vào khu vực không có khả năng tương tác trên thẻ sản phẩm.

2. Biến đầu vào: Thao tác vuốt (Swipe) để xóa sản phẩm **Miền hợp lệ (Valid Domains):**

- V3: Vuốt đúng trên một dòng sản phẩm đang tồn tại trong giỏ để xóa sản phẩm đó. **Miền không hợp lệ (Invalid Domains):**
- I3: Vuốt trên vùng trống khi giỏ hàng rỗng.

3. Biến trạng thái: Số lượng sản phẩm trong giỏ **Miền hợp lệ (Valid Domains):**

- V4: Số lượng = 1, đây là mức tối thiểu hợp lệ để sản phẩm còn hiển thị trong giỏ.
- V5: Số lượng > 1, hệ thống vẫn phải giữ sản phẩm trong giỏ và cập nhật tổng tiền tương ứng. **Miền không hợp lệ (Invalid Domains):**
- I4: Số lượng = 0, sản phẩm phải biến mất khỏi giỏ.

4. Biến trạng thái: Tổng tiền hiển thị trên UI **Miền hợp lệ (Valid Domains):**

- V6: Tổng tiền được tính lại ngay khi số lượng thay đổi. **Miền không hợp lệ (Invalid Domains):**
- I5: Tổng tiền không đổi sau khi số lượng đã thay đổi.

5. Biến trạng thái (Bổ sung): Trạng thái phiên đăng nhập (Authentication Session)

- Do giỏ hàng gắn liền với người dùng, sự tồn tại của sản phẩm trong giỏ phụ thuộc vào trạng thái login. **Miền hợp lệ (Valid Domains):**

- V7: Người dùng đang đăng nhập, xem giỏ hàng của chính mình. **Miền không hợp lệ / Biên rủi ro (Invalid/Edge Domains):**
- I6: Người dùng đã Đăng xuất (Logout), giỏ hàng trên máy phải được làm sạch (clear) để tránh lộ lọt dữ liệu hoặc thanh toán nhầm bởi người dùng tiếp theo.

Thiết kế Test Case (Domain Testing)

Dựa trên các miền giá trị trên, dưới đây là bộ Test Case đại diện để bao phủ các thao tác cảm ứng, trạng thái giỏ hàng và cập nhật số lượng trên mobile.

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-20_01	Kiểm tra chạm để chọn sản phẩm trong giỏ	Thiết bị: Mobile Giỏ hàng: 1 sản phẩm (30.000.000đ)	Chạm (Tap) vào dòng hiển thị sản phẩm.	Sản phẩm được chọn/đánh dấu, giỏ hàng không bị thay đổi số lượng hoặc tổng tiền.	Không có chuyện gì xảy ra. Số lượng và sản phẩm trong giỏ vẫn giữ nguyên bình thường.	Pass
TC_FR-20_02	Kiểm tra vuốt để xóa sản phẩm	Thiết bị: Mobile Giỏ hàng: 1 sản phẩm (30.000.000đ)	Vuốt (Swipe) và chọn nút "Xóa" trên dòng sản phẩm.	Hiện thị dialog xác nhận xóa. Sau khi xác nhận sản phẩm biến mất khỏi giỏ, tổng tiền về 0đ.	Sản phẩm bị xóa, tổng tiền về 0đ nhưng KHÔNG hiển thị dialog xác nhận xóa.	Fail
TC_FR-20_03	Kiểm tra tăng số lượng từ 1 lên 2	Thiết bị: Mobile Giỏ hàng: 1 sản phẩm Số lượng ban đầu: 1	Chỉnh sửa số lượng sản phẩm thành 2.	Số lượng hiển thị thành 2, sản phẩm vẫn còn trong giỏ, tổng tiền cập nhật thành 60.000.000đ.	UI thiếu nút "+/-". Nhập tay số "2" thì hệ thống tự cộng 1 thành "3", giá tiền sai thành 90.000.000đ.	Fail
TC_FR-20_04	Kiểm tra giảm số lượng từ 2 xuống 1	Thiết bị: Mobile Giỏ hàng: 1 sản phẩm Số lượng ban đầu: 2	Giảm số lượng sản phẩm xuống 1.	Số lượng hiển thị thành 1, sản phẩm vẫn còn trong giỏ, tổng tiền cập nhật thành 30.000.000đ.	Số lượng hiển thị thành 1, sản phẩm vẫn còn trong giỏ, tổng tiền cập nhật thành 30.000.000đ.	Pass
TC_FR-20_05	Kiểm tra giảm số lượng từ 1 xuống 0	Thiết bị: Mobile Giỏ hàng: 1 sản phẩm Số lượng ban đầu: 1	Cố tình giảm số lượng sản phẩm xuống 0.	Sản phẩm biến mất khỏi giỏ, hoặc hệ thống chặn không cho phép giảm về 0 (giữ nguyên là 1).	Số lượng hiển thị thành 1, sản phẩm vẫn còn trong giỏ (Hệ thống chặn giảm dưới 1).	Pass
TC_FR-20_06	Kiểm tra thao tác chạm vào vùng trống	Thiết bị: Mobile Giỏ hàng: Có sản phẩm	Chạm (Tap) vào vùng trống ngoài sản phẩm.	Hệ thống không chọn nhầm sản phẩm, không làm thay đổi tổng tiền.	Hệ thống không chọn nhầm sản phẩm, không thay đổi số lượng và không làm thay đổi tổng tiền.	Pass

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-20_07	Kiểm tra quản lý phiên (Đăng xuất)	Tài khoản A đăng nhập, giỏ hàng có 1 sản phẩm.	Thực hiện Đăng xuất khỏi hệ thống và xem lại trạng thái giỏ hàng.	Giỏ hàng được dọn sạch, giao diện quay về giỏ hàng trống hoặc yêu cầu đăng nhập.	Sau khi đăng xuất, truy cập lại giỏ hàng vẫn chứa nguyên sản phẩm của tài khoản cũ.	Fail

Boundary Value Analysis (BVA)

Phân tích các giá trị biên (Step-by-step Explanation):

Với FR-20, biên quan trọng nhất nằm ở số lượng sản phẩm trong giỏ. Đặc tả chỉ nêu rõ mức số lượng tối thiểu là 1, nên tôi chọn biên dưới làm trọng tâm: just below boundary là 0, on the boundary là 1, và just above boundary là 2. Vì đặc tả không có quy tắc tự động xóa khi số lượng bằng 0 (việc xóa phải qua nút chuyên dụng có dialog xác nhận), BVA sẽ tập trung kiểm tra xem hệ thống có ngăn chặn (block) thao tác giảm số lượng xuống dưới 1 một cách an toàn hay không, và việc tính toán tổng tiền có thay đổi chính xác tại các mức biên này không.

Thiết kế Test Case (BVA Testing):

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-20_BVA_01	Kiểm tra biên dưới số lượng (0)	Thiết bị: Mobile Số lượng ban đầu: 1	Thực hiện các thao tác trên UI/API để ép số lượng thành 0.	Không cho phép sản phẩm về 0 (giữ nguyên là 1) hoặc sản phẩm biến mất khỏi giỏ.	Số lượng hiển thị thành 1, sản phẩm vẫn còn trong giỏ (bị chặn ở mức 1).	Pass
TC_FR-20_BVA_02	Kiểm tra tại biên số lượng tối thiểu (1)	Thiết bị: Mobile Số lượng ban đầu: 1	Mở trang giỏ hàng và quan sát trạng thái.	Sản phẩm vẫn hiển thị trong giỏ, tổng tiền hiển thị đúng 30.000.000đ.	Sản phẩm vẫn hiển thị trong giỏ, tổng tiền hiển thị đúng 30.000.000đ.	Pass
TC_FR-20_BVA_03	Kiểm tra vượt biên số lượng tối thiểu (2)	Thiết bị: Mobile Số lượng ban đầu: 2	Mở trang giỏ hàng và quan sát trạng thái.	Sản phẩm vẫn hiển thị trong giỏ, tổng tiền hiển thị đúng 60.000.000đ.	Sản phẩm vẫn hiển thị trong giỏ, tổng tiền hiển thị đúng 60.000.000đ.	Pass
TC_FR-20_BVA_04	Kiểm tra vượt xóa khi sản phẩm đang tồn tại	Thiết bị: Mobile Giỏ hàng: Có sản phẩm	Bấm nút xóa trên dòng sản phẩm đang tồn tại.	Hiển thị dialog xác nhận xóa. Nếu đồng ý, tổng tiền về 0đ.	Sản phẩm biến mất khỏi giỏ ngay lập tức, tổng tiền về 0đ. KHÔNG hiện dialog xác nhận.	Fail

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data)	Thực hiện (Test step)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Verdict
TC_FR-20_BVA_05	Kiểm tra vuốt trên giỏ hàng rỗng	Thiết bị: Mobile Giỏ hàng: Trống	Vuốt (Swipe) trên vùng trống của giỏ hàng.	Hệ thống không bị lỗi giao diện, không crash và vẫn hiển thị trạng thái giỏ rỗng.	Hệ thống không bị lỗi giao diện, không crash và vẫn hiển thị trạng thái giỏ rỗng bình thường.	Pass

AI Gap Analysis

1. Bỏ quên biến trạng thái Phiên Đăng nhập (Authentication Session):

- AI chỉ tập trung vào các thao tác vật lý trên UI (chạm/vuốt) mà hoàn toàn quên mất Giỏ hàng phải được liên kết chặt chẽ với Session của người dùng. AI không thiết kế kịch bản cho trạng thái Đăng xuất, dẫn đến việc tôi phải tự bổ sung và phát hiện ra **Bug 19** (Rò rỉ giỏ hàng).
- **Lý do:** AI phân tích cục bộ trong phạm vi một màn hình Mobile UI mà thiếu góc nhìn tổng thể về kiến trúc State Management (Client-Server) của hệ thống.

2. AI tự hallucinate ra luồng xử lý số lượng bằng 0:

- Mặc dù thực tế điều này có thể đúng, nhưng đặc tả FR-20 không nói đến việc tự động xóa khỏi giỏ khi sửa số lượng bằng 0.
- **Lý do:** Có lẽ AI đã tự dựa vào thường thức phổ thông (common sense) của các hệ thống E-commerce có sẵn trong tập huấn luyện để áp đặt tính năng.

3. Thiếu thông tin về kiểu vuốt và vùng tương tác chính xác của UI:

- Đặc tả nói có swipe để xóa và tap để chọn, nhưng không mô tả rõ là vuốt trái hay phải, có hiển thị hitbox hay không.
- **Lý do:** Khi thực thi thực tế trên thiết bị thật, hành vi gesture có thể khác nhau và AI không đủ ngữ cảnh để lường tượng ra UI thực tế nếu không có ảnh mockup. Việc này dẫn đến việc không lường trước được sự cố UI thiếu nút +/- (Bug 16).

4. AI bỏ qua đặc tả về Dialog xác nhận xóa:

- AI hoàn toàn bỏ qua đặc tả rằng khi xóa sản phẩm khỏi giỏ hàng thì phải hiện dialog xác nhận. Testcase ban đầu mà AI cung cấp cho rằng khi xóa là sản phẩm biến mất ngay lập tức (Dẫn đến hụt mất Bug 17).
- **Lý do:** AI có thể đã quét nhanh và bỏ sót một câu yêu cầu nhỏ trong tài liệu đặc tả thô, hoặc tự cho rằng bước confirm dialog là không quan trọng đối với thao tác swipe.

Bug Reporting

Toàn bộ danh sách lỗi phát hiện được từ các Test Case trên đã được báo cáo chi tiết kèm ảnh chụp màn hình và link GitHub Issues trong file đính kèm [Bug_Report.md](#)